Algoritmo de *Work Stealing*

Octubre/2026



Contenido

- Preámbulo
  - Mecanismos de concurrencia con hilos en Java
- Estructura del artículo
- Introducción
- Un modelo de computación multihilos
- La propiedad de “hojas ocupadas”
- Un algoritmo de robo de trabajo (work-stealing) aleatorizado
- Acceso atómico y el juego de reciclaje
- Análisis del algoritmo de robo de trabajo (work-stealing)
- Otros artículos para revisar
- Conclusiones y recomendaciones

Programa mutihilo

- Un programa mutihilo, cuando se ejecuta sobre un conjunto de datos de entrada (*input*), en ocasiones puede generar más de una computación multihilos.
- En este caso, decimos que el programa es **NO determinístico**.
- Si la misma computación multihilo es creada por el programa que procesa cierta entrada sin importar como sea planificada, entonces el programa es **determinístico**.
- EN ESTE ARTÍCULO SE ANALIZAN COMPUTACIONES MULTIHILO, NO PROGRAMAS MULTIHILO
- De manera específica, el artículo NO se preocupa sobre cómo las computaciones multihilo son generadas.
- En su lugar, se estudiarán sus propiedades "*a posteriori*"

Computación multihilo estricta

- Debido a que las computaciones multihilo con dependencias arbitrarias es imposible planificarlas eficientemente, aquí se estudia una subclase de computaciones multihilo en las cuáles los tipos de sincronización que pueden ocurrir están restringidos.
- Una computación multihilo estricta es aquella en la cual todos los arcos de unión que salen de un hilo van a un ancestro dentro del árbol de generación.
- En una computación estricta el único arco hacia un sub-árbol (que se origina fuera del sub-árbol) es el arco de creación que genera el hilo raíz del sub-árbol [ver ejemplo].
- El adjetivo “estricto” significa que un hilo no puede ser invocado antes que todos sus argumentos estén disponibles, aunque sus argumentos pueden ser creados de manera paralela.



Computación multihilo completamente estricta

- Una computación completamente estricta es aquella donde todos los arcos de unión de un hilo, van al hilo padre.
- En este sentido una computación completamente estricta es una computación “bien estructurada” pues todos los arcos de unión de un sub-árbol (del árbol de generación/creación) emanan de la raíz del sub-árbol.

- El ejemplo de este artículo es una computación completamente estricta
  - Cualquier computación que pueda ser ejecutada de la manera Depth-first (primero en profundidad) sobre un único procesador puede hacerse estricta o completamente estricta alterando la estructura de dependencia, donde se puede afectar el paralelismo obtenido, pero no se afecta la semántica de la computación.

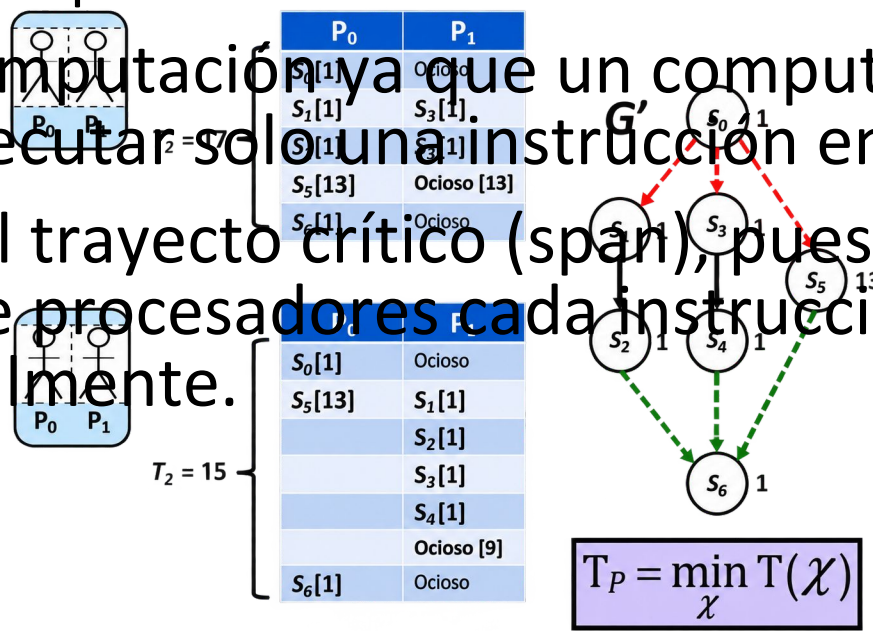


Work y Span (*critical-path length*)

- El tiempo de ejecución de una computación realizada en P procesadores se puede cuantificar y acotar en términos del "Work" y el "Span" (*critical-path length*)
- Para este modelo se define work como el número total de instrucciones [recordar que cada instrucción se ejecuta en una unidad de tiempo]
- Y el span(*critical-path length*) como la longitud del trayecto dirigido más largo en el dag.
- Para el ejemplo el work es 23 y el span es 10.

Work y Span (*critical-path length*)

- Para una computación dada, sea T(χ) el tiempo para ejecutar la computación utilizando la planificación de ejecución χ para P procesadores
  - Sea $T_P = \min_\chi T(\chi)$ el tiempo de ejecución mínimo con P procesadores
  - El mínimo se escoge del conjunto de todas las planificaciones de ejecución para P procesadores de la computación
- T_1 es el work de la computación ya que un computador con un único procesador puede ejecutar solo una instrucción en cada paso
- y T_∞ es la longitud del trayecto crítico (span), pues incluso con una cantidad arbitraria de procesadores cada instrucción dentro del trayecto debe ejecutarse serialmente.



Work y Span (*critical-path length*)

- Nótese que se tiene que $T_P \geq T_1/P$ debido a que P procesadores pueden ejecutar solo P instrucciones por unidad de tiempo
- También se tiene que $T_P \geq T_\infty$
- Las primeras investigaciones sobre planificación en dag probó que existen planificaciones de ejecución χ para P procesadores con $T(\chi) \leq T_1/P + T_\infty$
- Esta cota, como la suma de dos límites inferiores, es universalmente óptima.

Planificadores codiciosos

- Planificadores codiciosos (*greedy schedules*): aquellos en los cuales cada paso de ejecución, si al menos P instrucciones están listas, entonces P instrucciones se ejecutarán, y si se tienen menos de P instrucciones listas (estado *ready*), entonces todas son ejecutadas.
  - Es decir, se debe ejecutar todo lo que esté listo para ejecutarse en los P procesadores que se tienen…



Teorema 1

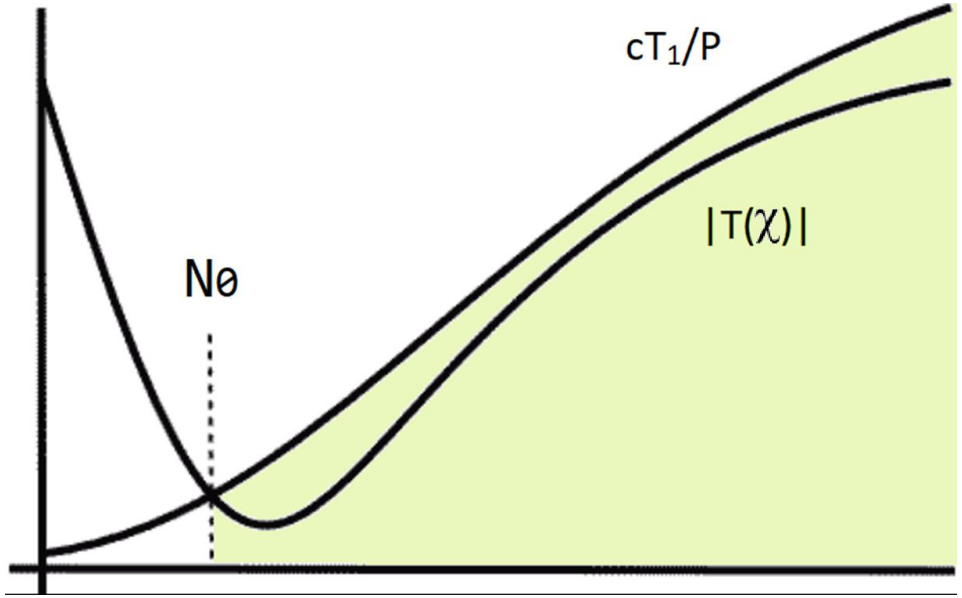
- Para cualquier computación multihilo con Work T_1 y Span T_∞; y para cualquier número P de procesadores, cualquier planificación de ejecución codiciosa χ para P procesadores alcanza

$$T(\chi) \leq T_1/P + T_\infty$$

$$T(\chi) \leq \mathit{Work}/P + \mathit{Span}$$

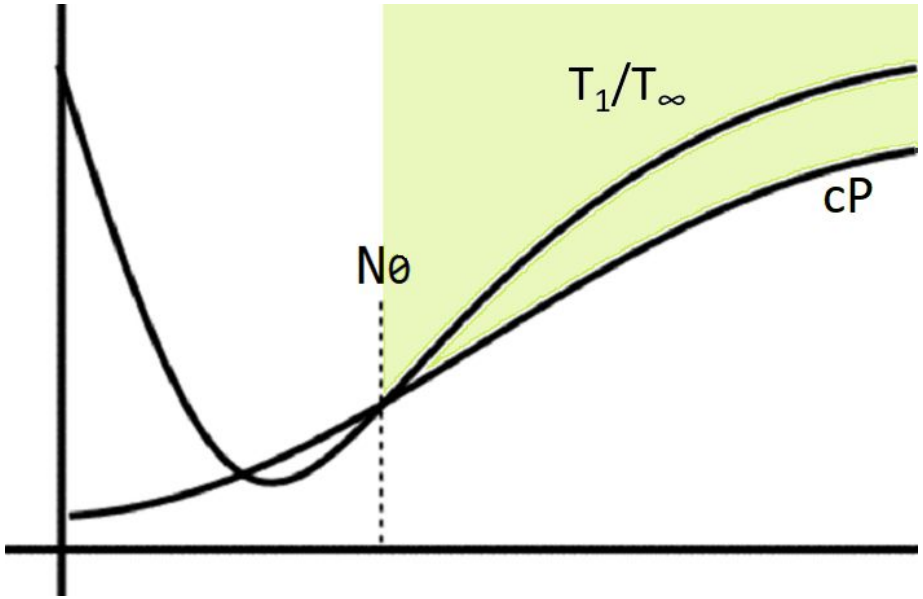
Speedup lineal

- En general estaremos interesados en planificadores que logren mejoras en la rapidez de computación lineales.
- Es decir $T(\chi) = O(T_1/P)$ [léase $T(\chi) \in O(T_1/P)$]



Speedup lineal para planificación codiciosa

- En planificación codiciosa (*greedy schedule*), la mejora en la rapidez de computación lineal (*linear speedup*) ocurre cuando el paralelismo, que se define aquí como T_1/T_∞ satisface que $T_1/T_\infty = \Omega(P)$ [léase $T_1/T_\infty \in \Omega(P)$]



Propiedad de “hojas ocupadas” (*busy leaves*)

- Una vez un hilo Γ ha sido generado en una computación estricta, un único procesador puede completar la ejecución de la subcomputación con raíz en Γ, incluso si no hay progreso en otras partes de la computación.
  - En otras palabras, desde el momento que el hilo Γ es generado hasta que el hilo Γ muere, se tiene siempre al menos un hilo de la subcomputación con raíz en Γ que está en estado *ready*.
  - En particular, ningún hilo hoja en una computación multihilo estricta puede suspenderse (*stall*)
  - Esta propiedad permite a una planificación de ejecución mantener las “hojas ocupadas” (“*busy*”)
- Al combinar la propiedad de “hojas ocupadas” con la propiedad de planificación codiciosa (*greedy*), se pueden derivar planificaciones de ejecución que exhiben simultáneamente mejora en la rapidez de la computación lineal (*linear speedup*) y expansión lineal del espacio.

$$T_1/T_\infty = \Omega(P)$$

$$S(\chi) = O(S_1 P)$$

Propiedad de “hojas ocupadas” (*busy leaves*)

- En la sección tres del artículo, se revisará que para cualquier número de procesadores P, y cualquier computación multihilo estricta con work T_1, span T_∞, tamaño de pila S_1, hay una planificación de ejecución χ para P procesadores que logra tener simultáneamente un tiempo $T(\chi) \leq T_1/P + T_\infty$ y un espacio $S(\chi) \leq S_1 P$
- Se presentará un algoritmo “simple” paralelo para P procesadores “*online*” que permite estimar dicha planificación: el algoritmo de hojas ocupadas (*Busy-Leaves Algorithm*)
- Este algoritmo forma la base para el algoritmo de robo de trabajo aleatorizado (*randomized work-stealing algorithm*) presentado en la sección 4.

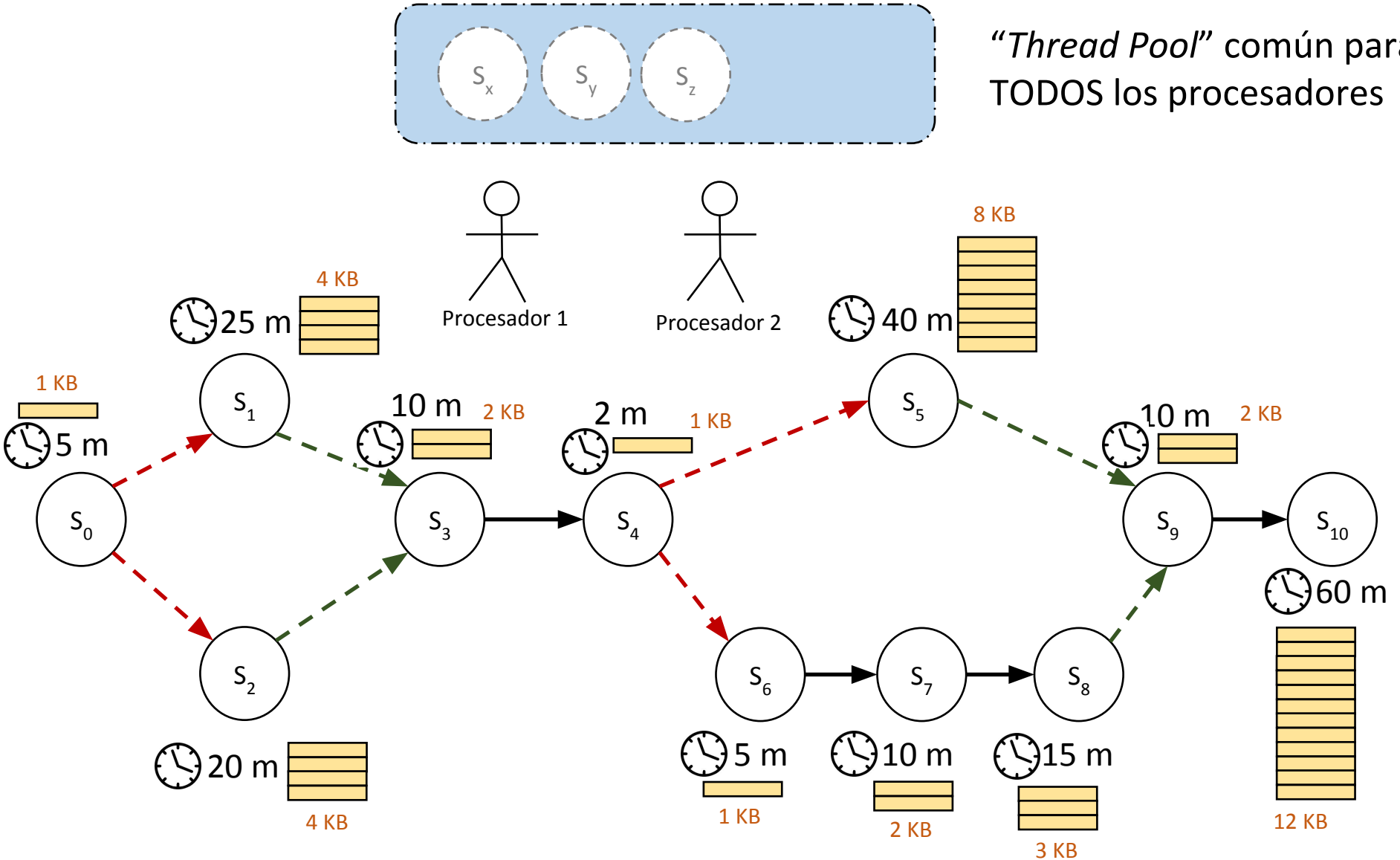
El algoritmo de hojas ocupadas es "*online*"

- Este algoritmo de considera "*online*" en el siguiente sentido:
  - Antes del t-ésimo paso, el algoritmo ha computado y ejecutado los primeros t-1 pasos de la planificación de la ejecución.
  - Para calcular y ejecutar el t-ésimo paso del planificador, el algoritmo solo utiliza información de la porción de la computación ejecutada hasta ese momento.
  - En particular, el algoritmo NO utiliza información de las instrucciones que aún no han sido ejecutadas o hilos que aún no han sido generados.

“*Pool*” global en el algoritmo de hojas ocupadas

- El algoritmo de hojas ocupadas mantiene todos los hilos vivos en un único “*pool*” de hilos que está disponible de manera uniforme para todos los P procesadores.
- Cuando se crean hilos, nuevos hilos son agregados al “*pool*” global, y cuando un procesador necesita trabajo, este saca del “*pool*” un hilo que está en estado *ready*.
  - Aunque el artículo describe el algoritmo de hojas ocupadas como un algoritmo paralelo para P procesadores, no será analizado como tal.
- Específicamente, al computar el t-ésimo paso, de la planificación, en el modelo se le permitirá a cada procesador agregar hilos y quitar (borrar) hilos del *pool* a su antojo.
  - De esta manera se ignorará el impacto de la contención de los procesadores para acceder el “*pool*”
- Solo se analizarán las propiedades del planificador y se ignorará el costo en que incurre el algoritmo al computar el planificador.

Grafo computacional para el algoritmo *"Busy-Leaves"*



Operación del algoritmo de “hojas ocupadas”

- El algoritmo comienza con el hilo raíz dentro del “*pool*” de hilos global y con todos los procesadores ociosos.
- Al inicio de cada paso, cada procesador o está ocioso o está ejecutando una instrucción de un hilo.
  - Al iniciar un paso, los procesadores que están ociosos intentan sacar del “*pool*” un hilo que esté en estado *ready*
  - Si hay una cantidad suficiente de hilos en estado *ready* en el “*pool*” global para poner a trabajar todos los procesadores que están ociosos, entonces todos ellos lograrán ocuparse. En otro caso, algunos procesadores permanecerán ociosos.
  - Para los procesadores que ya tengan un hilo, ejecutarán la siguiente instrucción de dicho hilo.

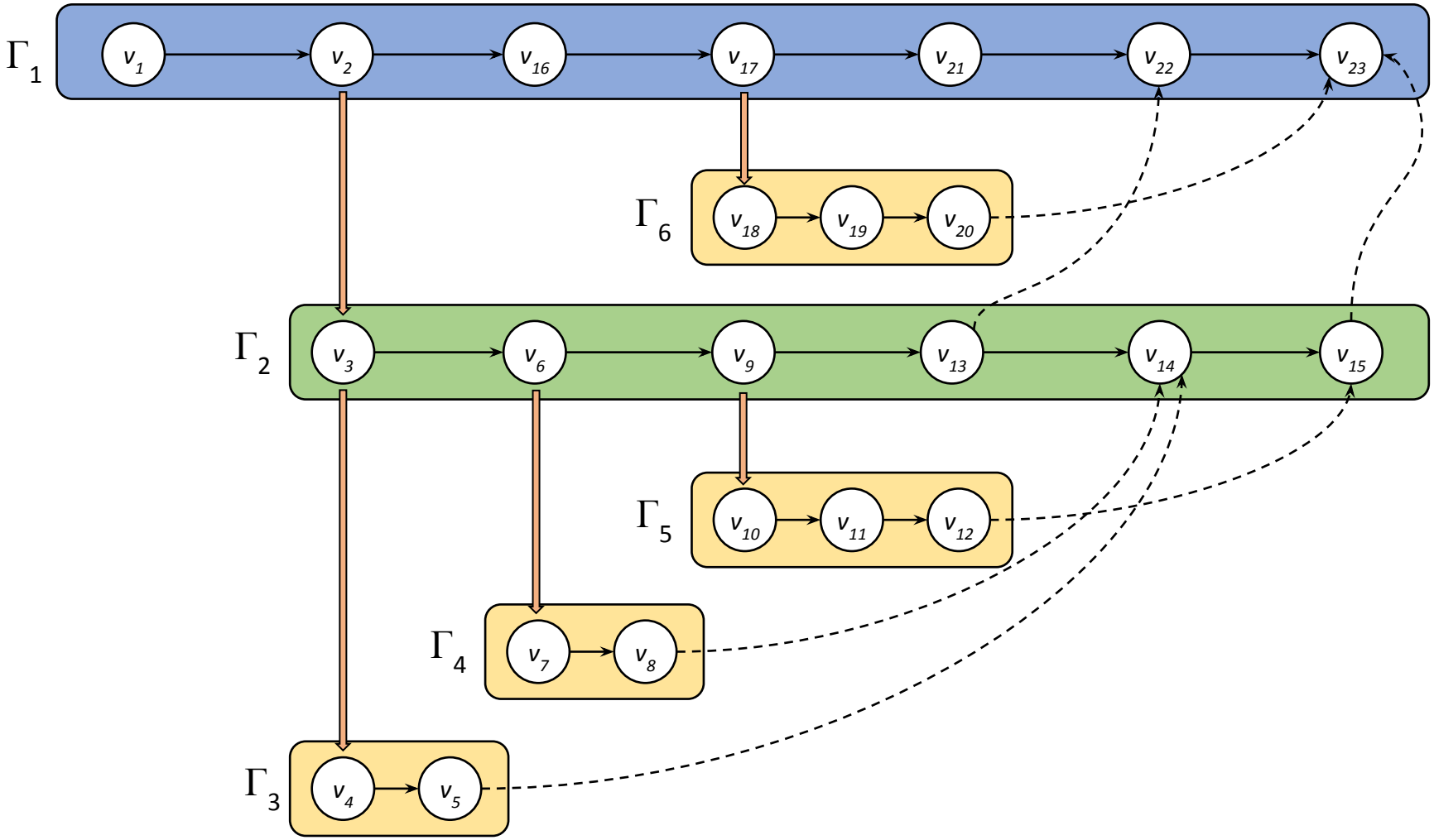
Operación del algoritmo de “hojas ocupadas”

- Una vez un procesador tenga un hilo, llamémoslo Γ_a, para trabajarlo, este ejecutará una instrucción de Γ_a en cada paso hasta que el hilo tenga una de tres situaciones
  1. Genere otro hilo (*spawn*)
  2. Se postergue (*stall*)
  3. Muera (*die*)
- Para cada situación, en el algoritmo, se respetarán ciertas reglas

Reglas de cambio de estado de los hilos

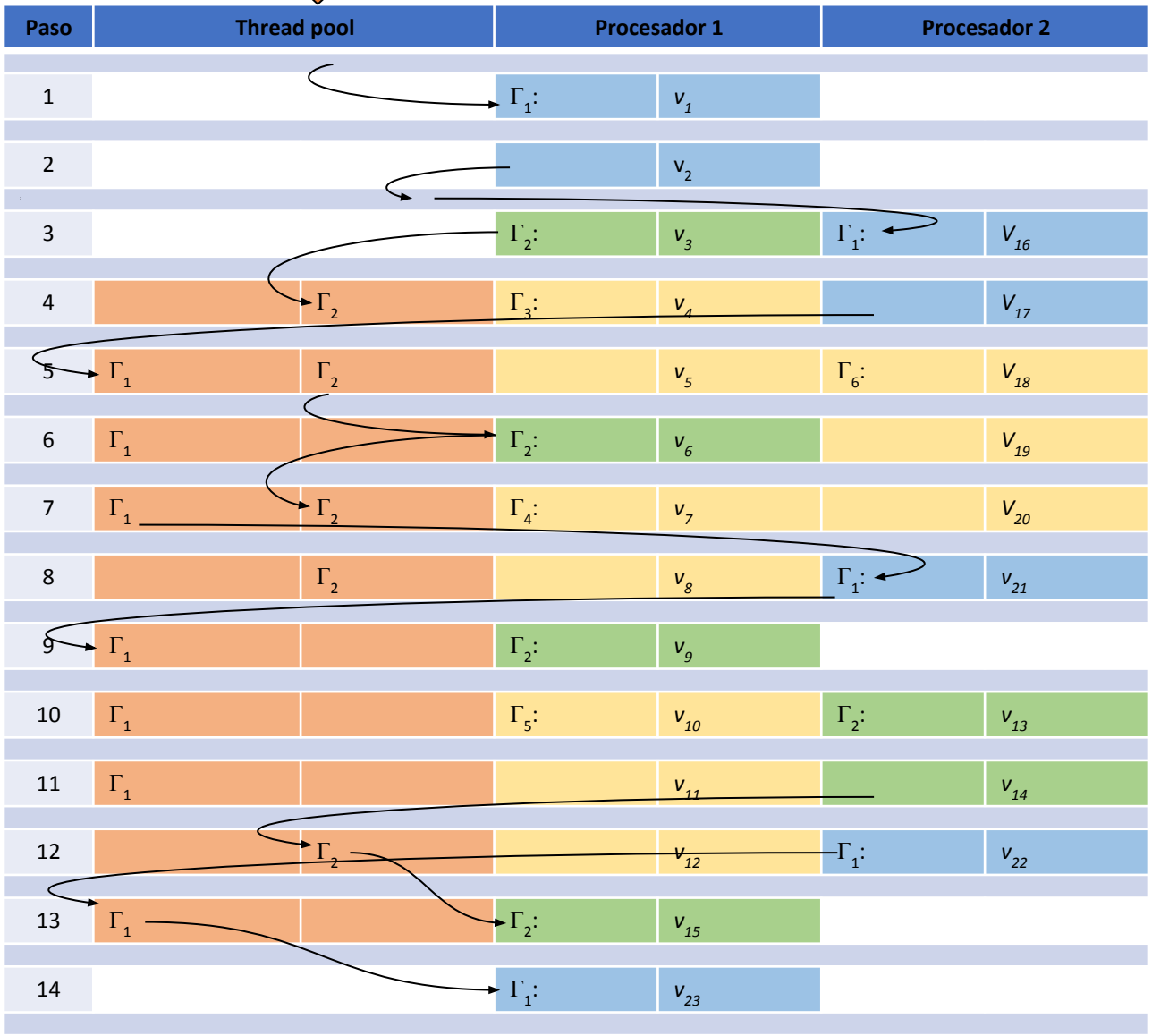
1. ***Spawns***: Si el hilo Γ_a crea o genera un hijo Γ_b, entonces el procesador termina el paso actual regresando el hilo Γ_a al "*pool*" global de hilos. El procesador inicia el siguiente paso trabajando sobre Γ_b.
2. ***Stalls***: Si el hilo Γ_a se posterga, se suspende, entonces el procesador concluye el paso actual retornando Γ_a al "*pool*" de hilos. El procesador inicia el siguiente paso ocioso.
3. ***Dies***: Si el hilo Γ_a muere, el procesador concluye el paso actual revisando si el padre de Γ_a, Γ_b, tiene algún otro hijo vivo.
   a. Si Γ_b NO tiene hijos vivos y ningún otro procesador está trabajando Γ_b, entonces el procesador saca Γ_b del "*pool*" y comienza el siguiente paso trabajando Γ_b.
   b. En otro caso, el procesador inicia el siguiente paso ocioso

Computación multihilo con 6 hilos y 23 instrucciones



Dos procesadores con el algoritmo “Busy-Leaves”

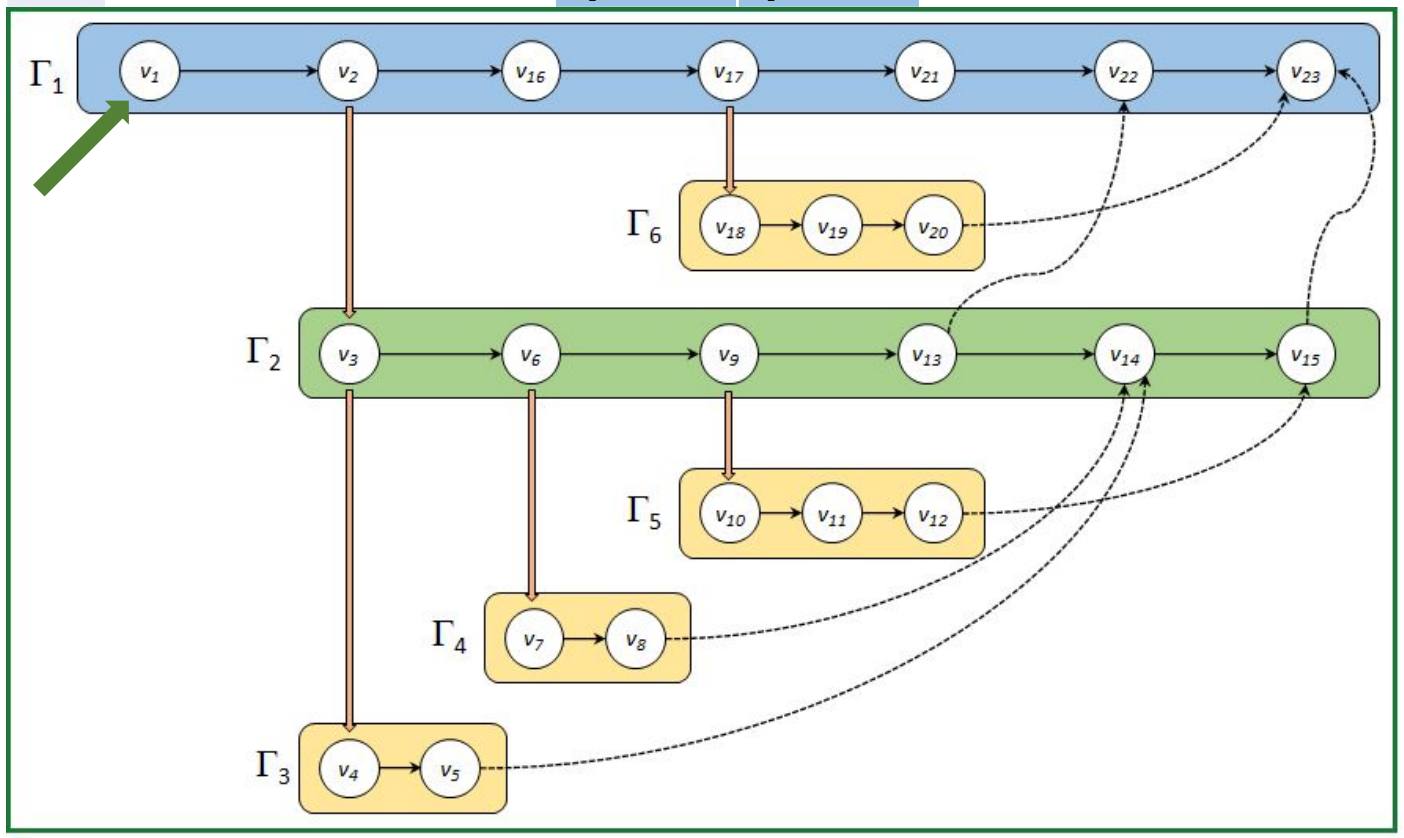
El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.



Dos procesadores con el algoritmo “Busy-Leaves”

El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.

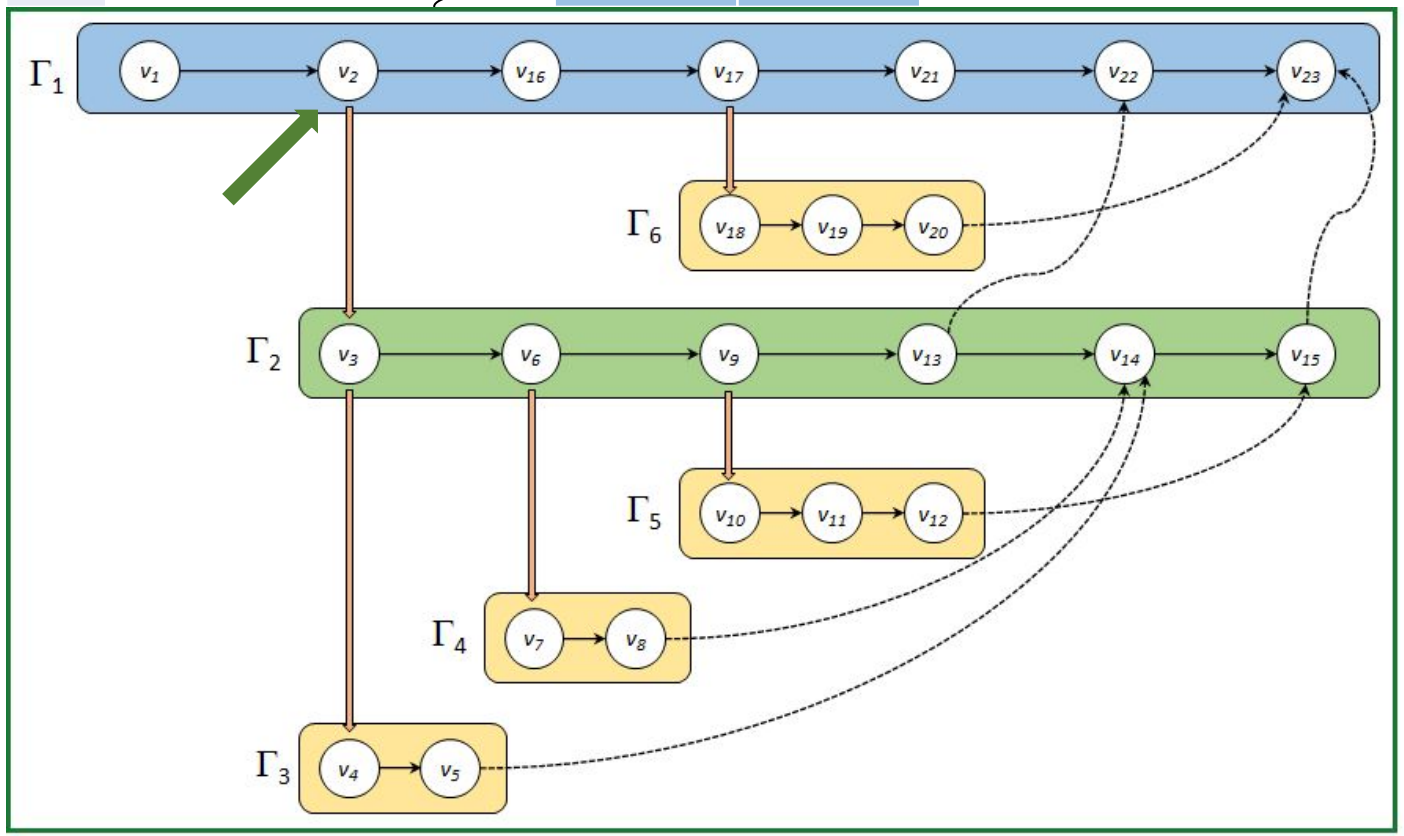
Paso	Thread pool	Procesador 1	Procesador 2
1		Γ_1:	v_1



Dos procesadores con el algoritmo “Busy-Leaves”

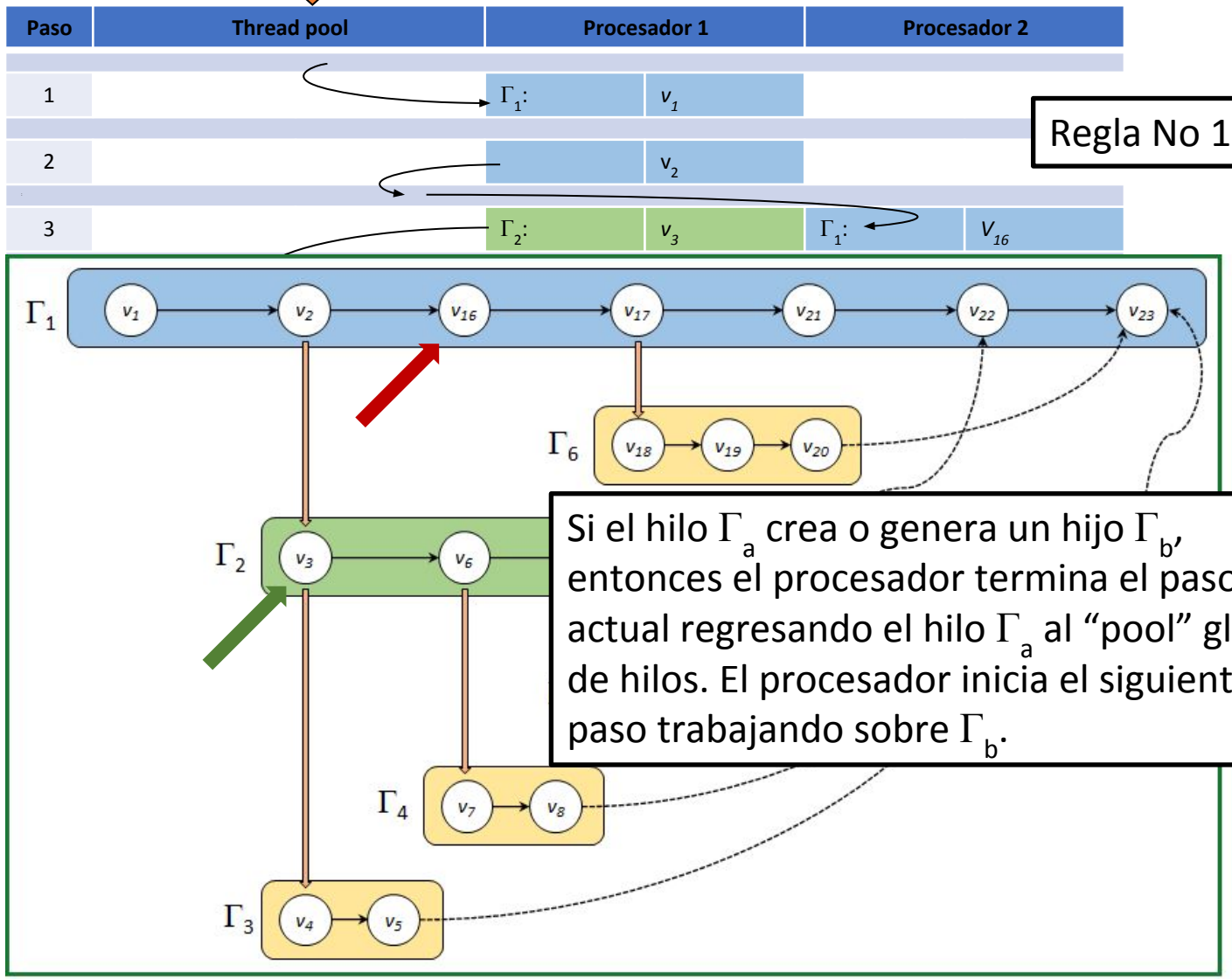
El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.

Paso	Thread pool	Procesador 1	Procesador 2
1		Γ_1:	v_1
2			v_2



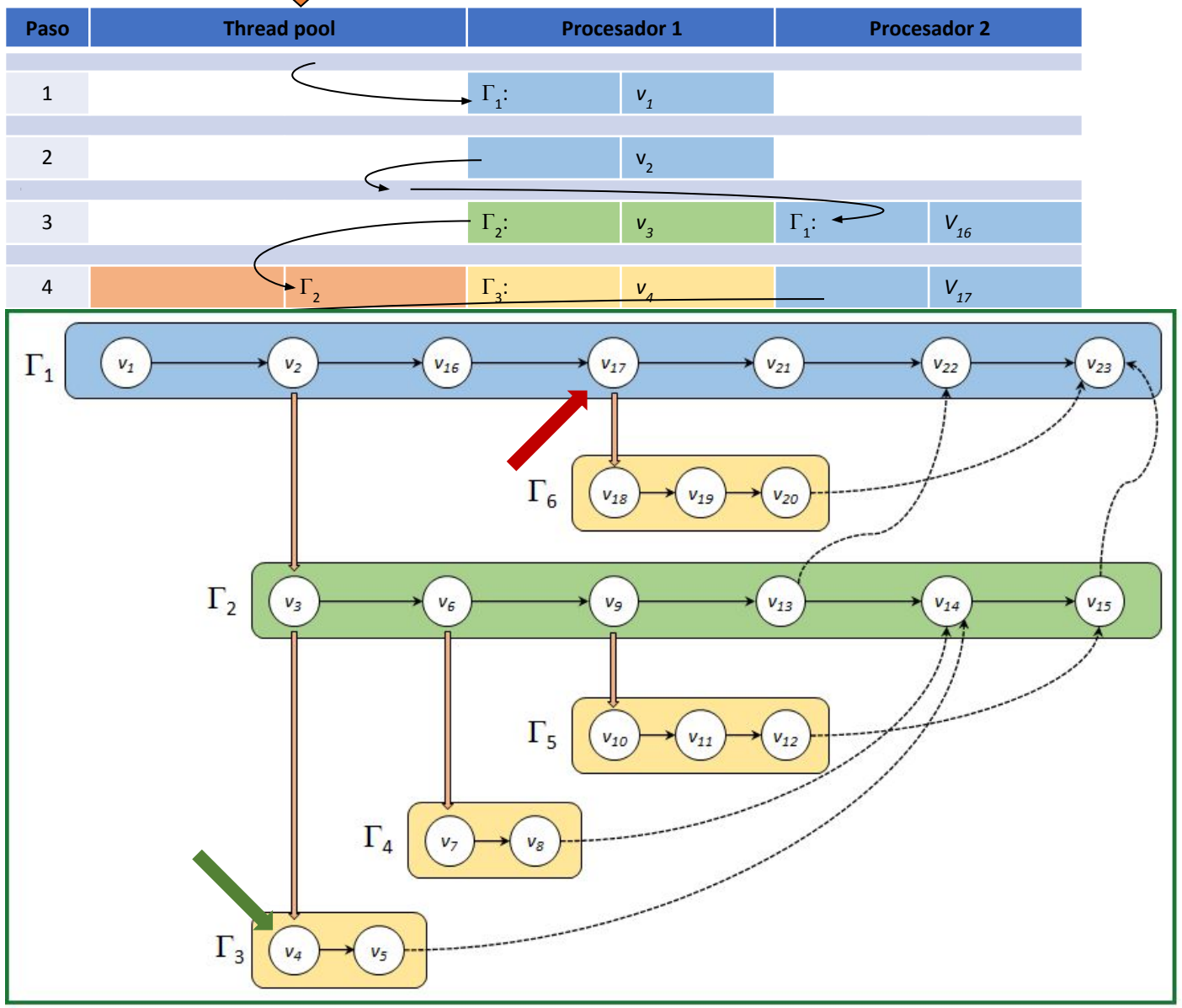
Dos procesadores con el algoritmo “Busy-Leaves”

El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.



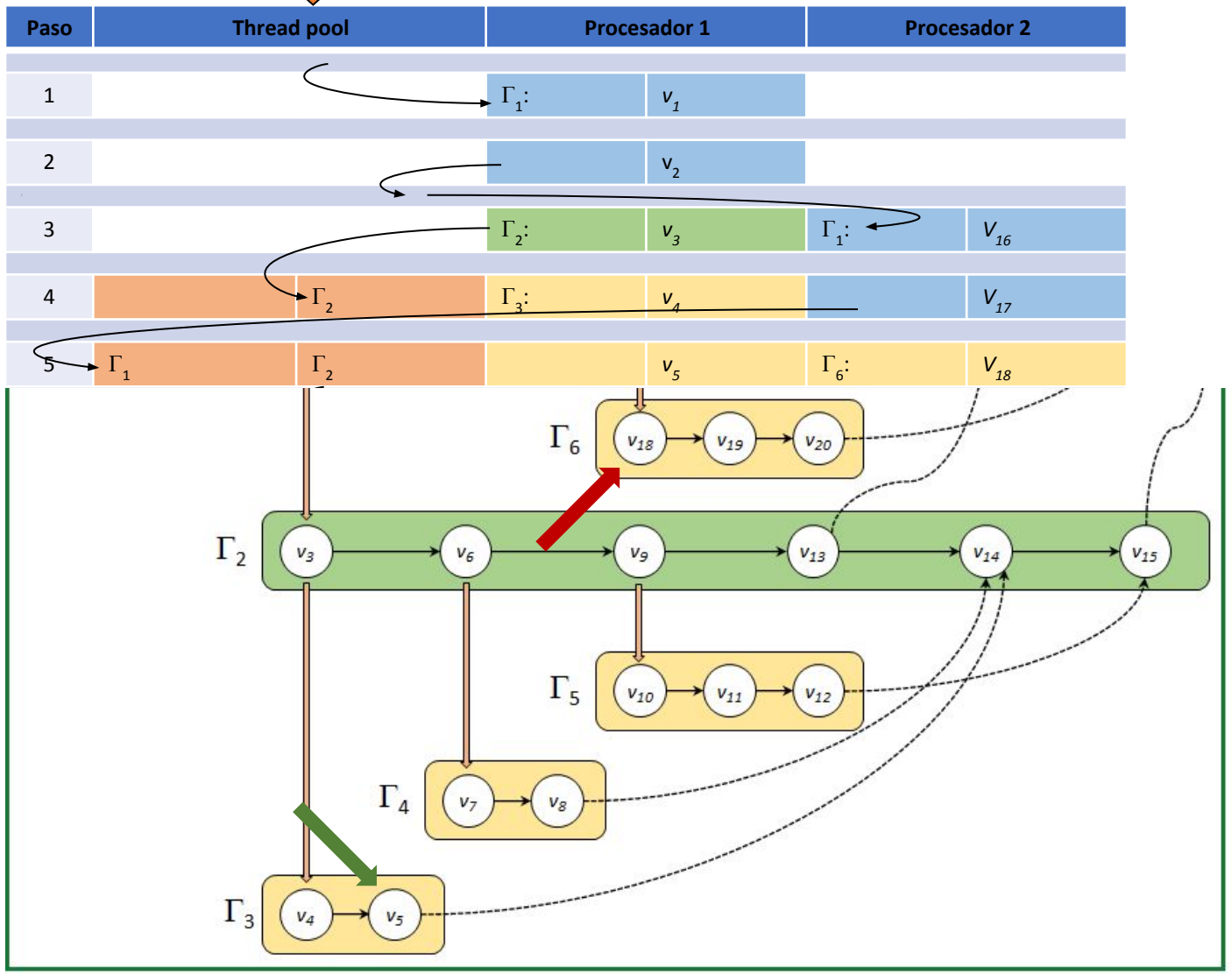
Dos procesadores con el algoritmo “Busy-Leaves”

El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.



Dos procesadores con el algoritmo “Busy-Leaves”

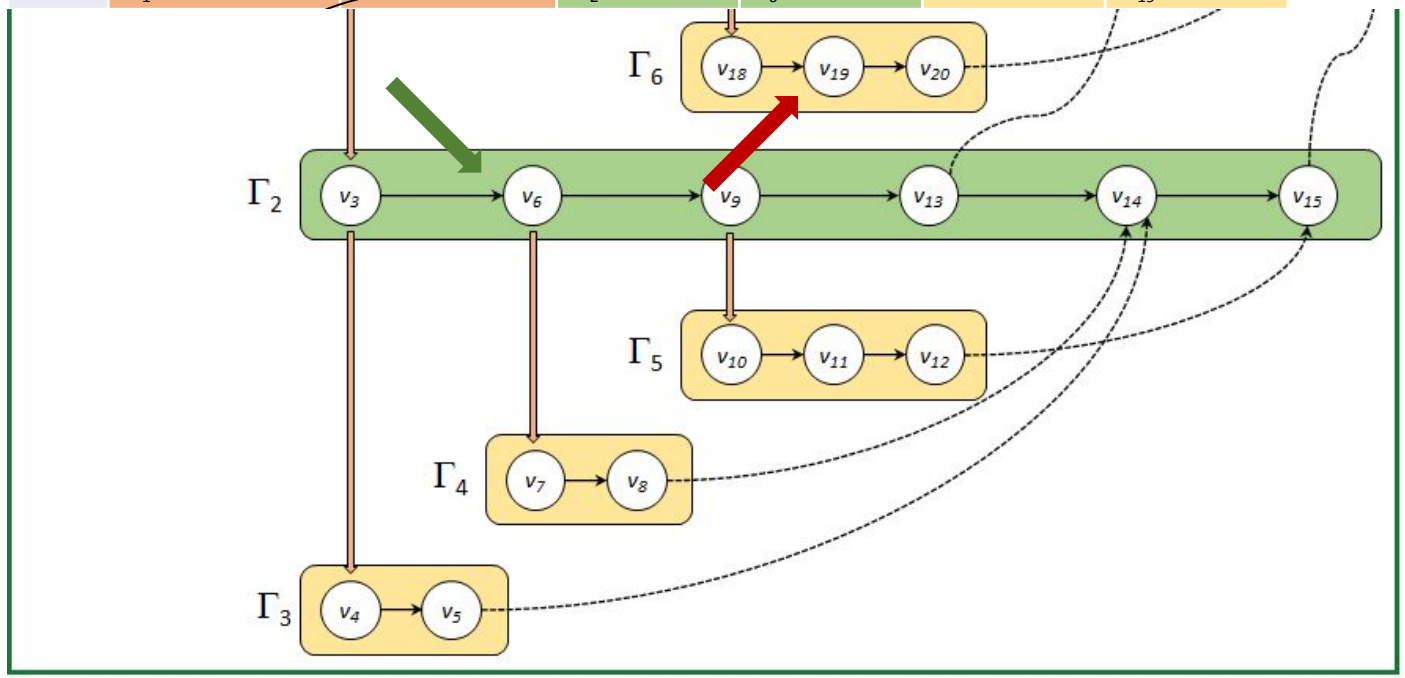
El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.



Dos procesadores con el algoritmo “Busy-Leaves”

El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.

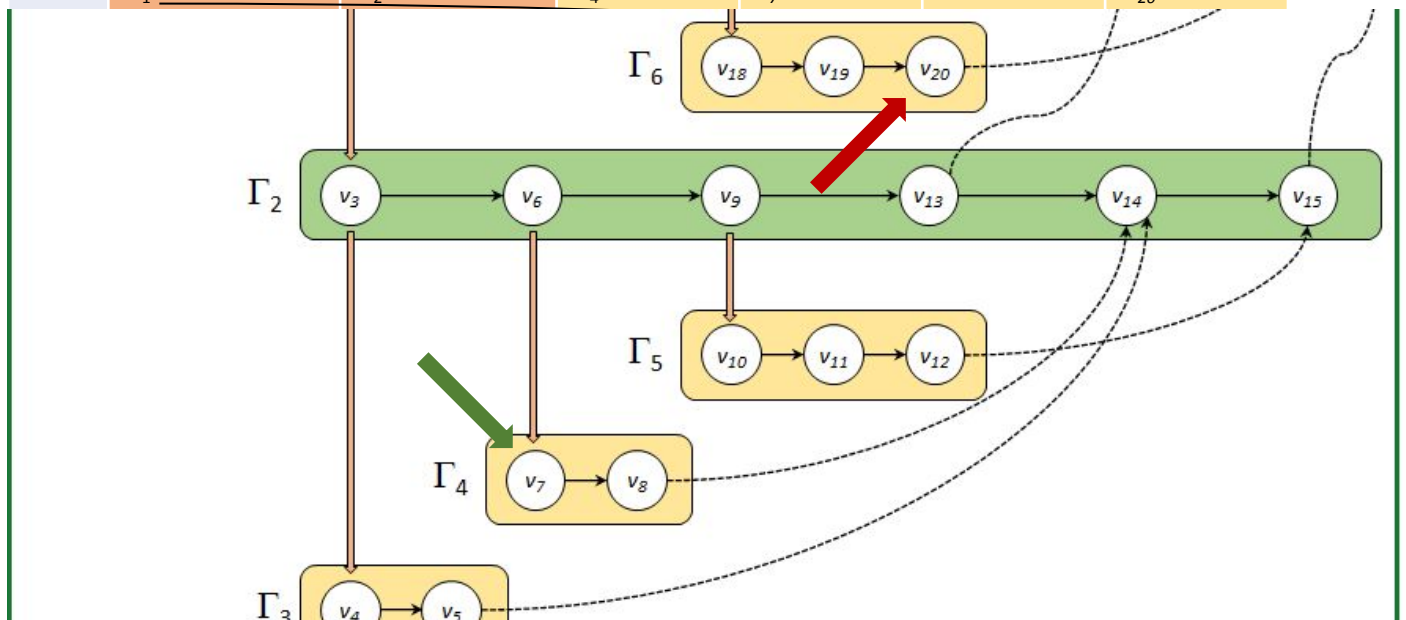
Paso	Thread pool	Procesador 1	Procesador 2
1		Γ_1: v_1	
2		v_2	
3		Γ_2: v_3	Γ_1: V_{16}
4	Γ_2	Γ_3: v_4	V_{17}
5	Γ_1 Γ_2	v_5	Γ_6: V_{18}
6	Γ_1	Γ_2: v_6	V_{19}



Dos procesadores con el algoritmo “Busy-Leaves”

El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.

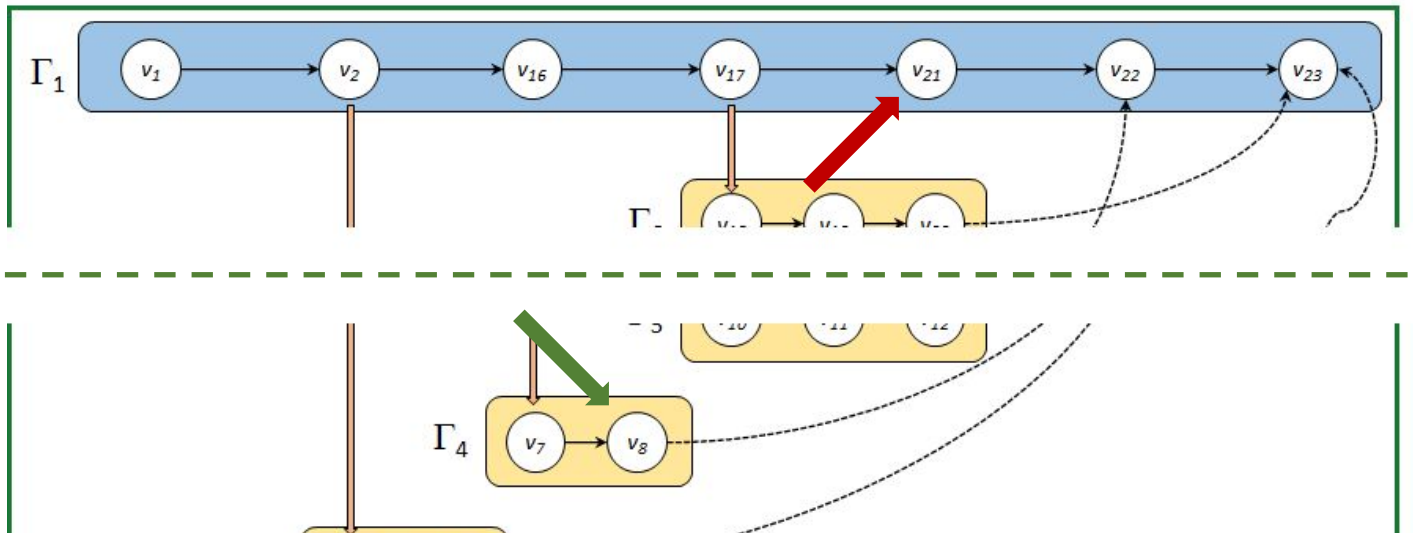
Paso	Thread pool	Procesador 1	Procesador 2
1		Γ_1: v_1	
2		v_2	
3		Γ_2: v_3	Γ_1: V_{16}
4	Γ_2	Γ_3: v_4	V_{17}
5	Γ_1 Γ_2	v_5	Γ_6: V_{18}
6	Γ_1	Γ_2: v_6	V_{19}
7	Γ_1 Γ_2	Γ_4: v_7	V_{20}



Dos procesadores con el algoritmo “Busy-Leaves”

El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.

Paso	Thread pool	Procesador 1	Procesador 2
1		Γ_1: v_1	
2		v_2	
3		Γ_2: v_3	Γ_1: V_{16}
4	Γ_2	Γ_3: v_4	V_{17}
5	Γ_1 Γ_2	v_5	Γ_6: V_{18}
6	Γ_1	Γ_2: v_6	V_{19}
7	Γ_1 Γ_2	Γ_4: v_7	V_{20}
8	Γ_2	v_8	Γ_1: v_{21}

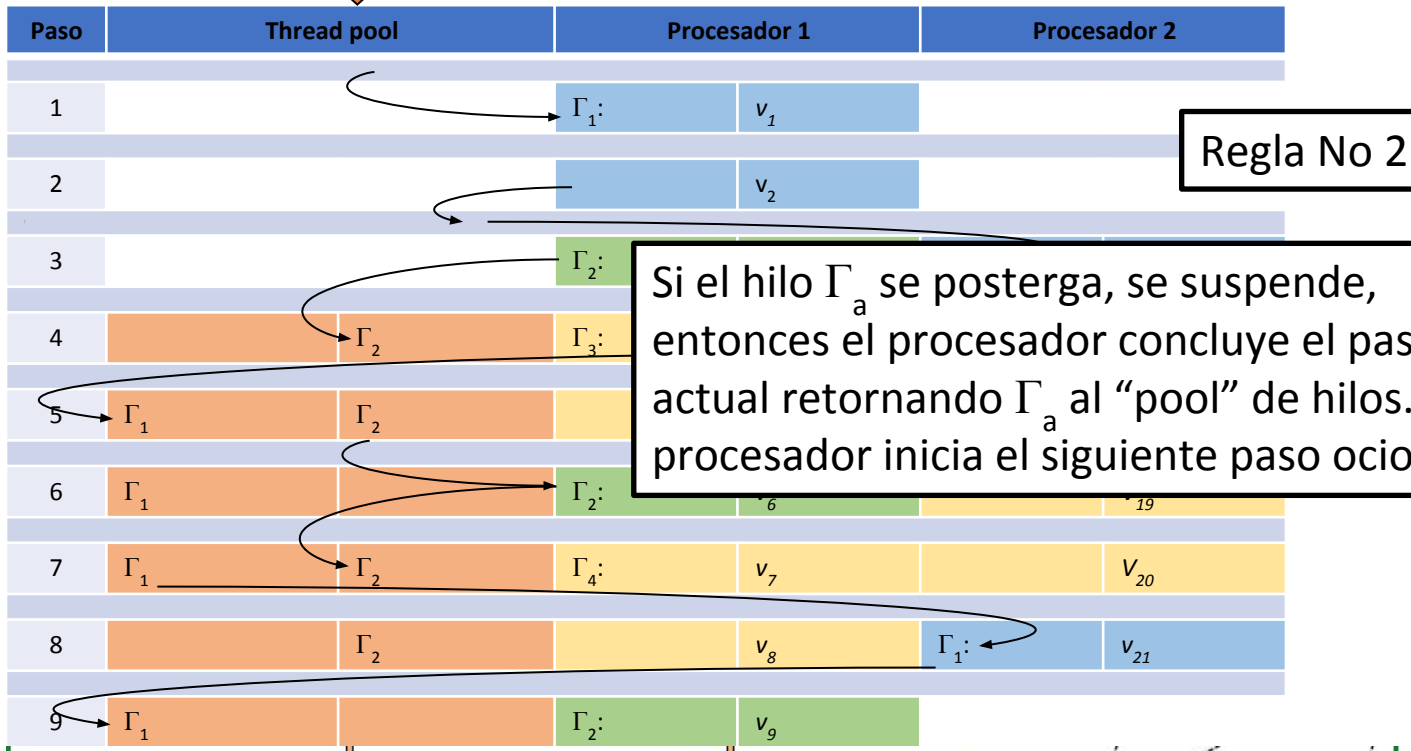


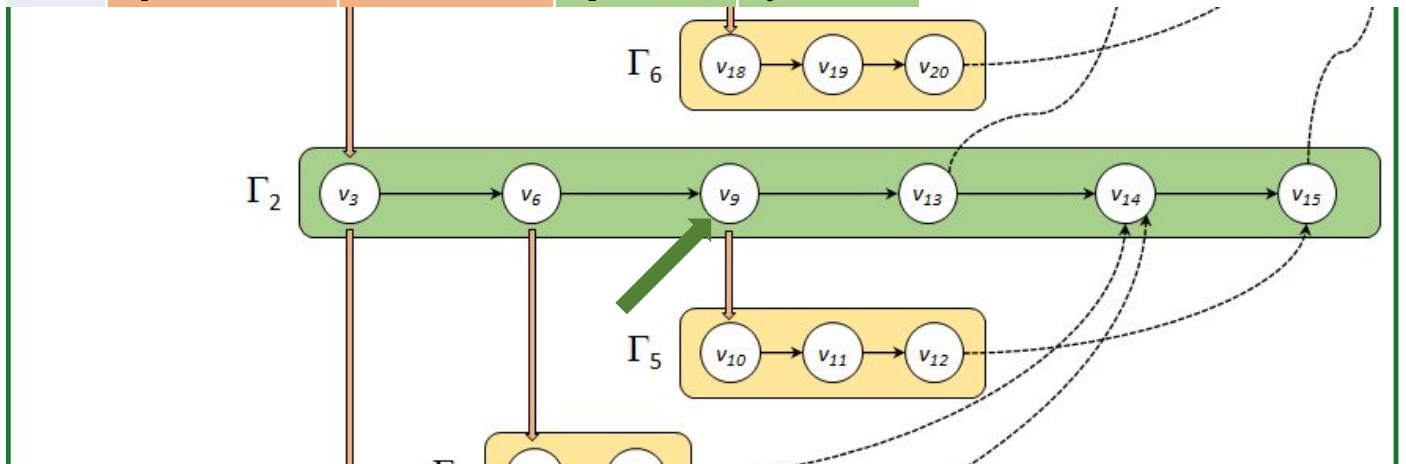
Dos procesadores con el algoritmo “Busy-Leaves”

El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.

Regla No 2

Si el hilo Γ_a se posterga, se suspende, entonces el procesador concluye el paso actual retornando Γ_a al “pool” de hilos. El procesador inicia el siguiente paso ocioso.

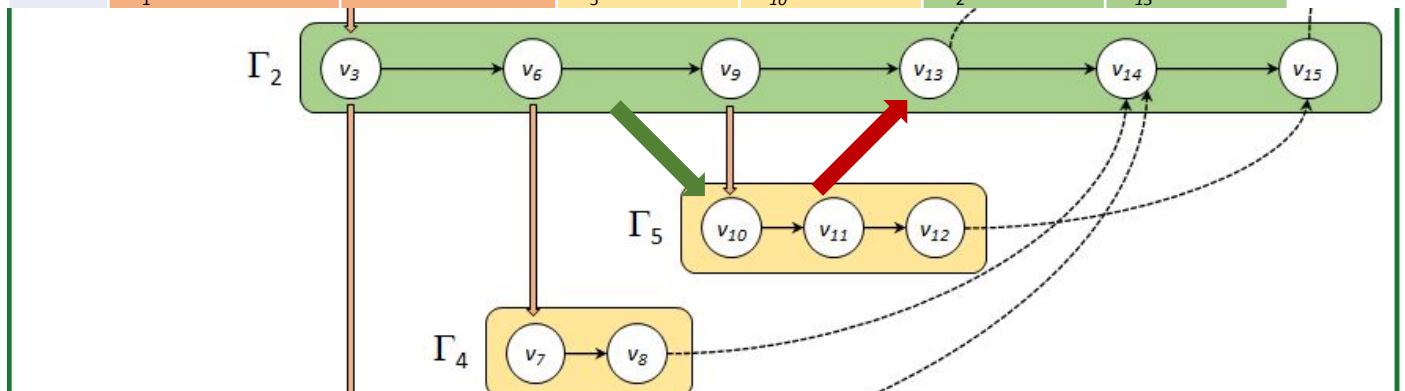




Dos procesadores con el algoritmo “Busy-Leaves”

El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.

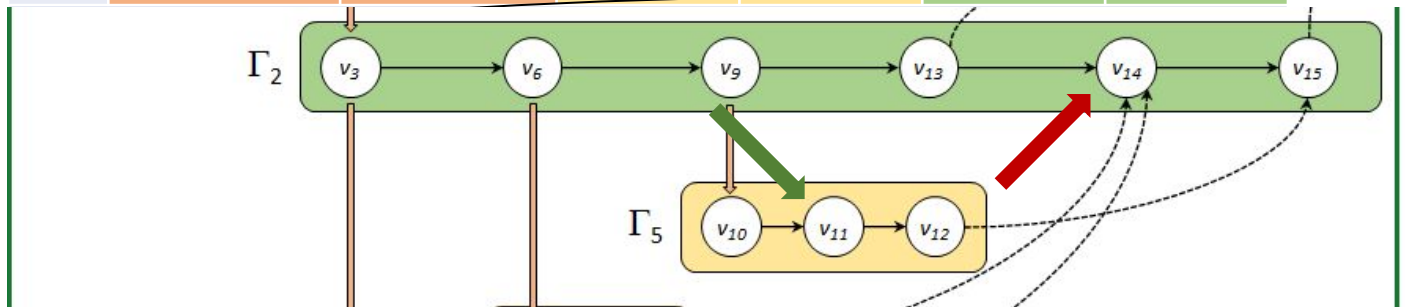
Paso	Thread pool	Procesador 1	Procesador 2
1		Γ_1: v_1	
2		v_2	
3		Γ_2: v_3	Γ_1: V_{16}
4	Γ_2	Γ_3: v_4	V_{17}
5	Γ_1 Γ_2	v_5	Γ_6: V_{18}
6	Γ_1	Γ_2: v_6	V_{19}
7	Γ_1 Γ_2	Γ_4: v_7	V_{20}
8	Γ_2	v_8	Γ_1: v_{21}
9	Γ_1	Γ_2: v_9	
10	Γ_1	Γ_5: v_{10}	Γ_2: v_{13}



Dos procesadores con el algoritmo “Busy-Leaves”

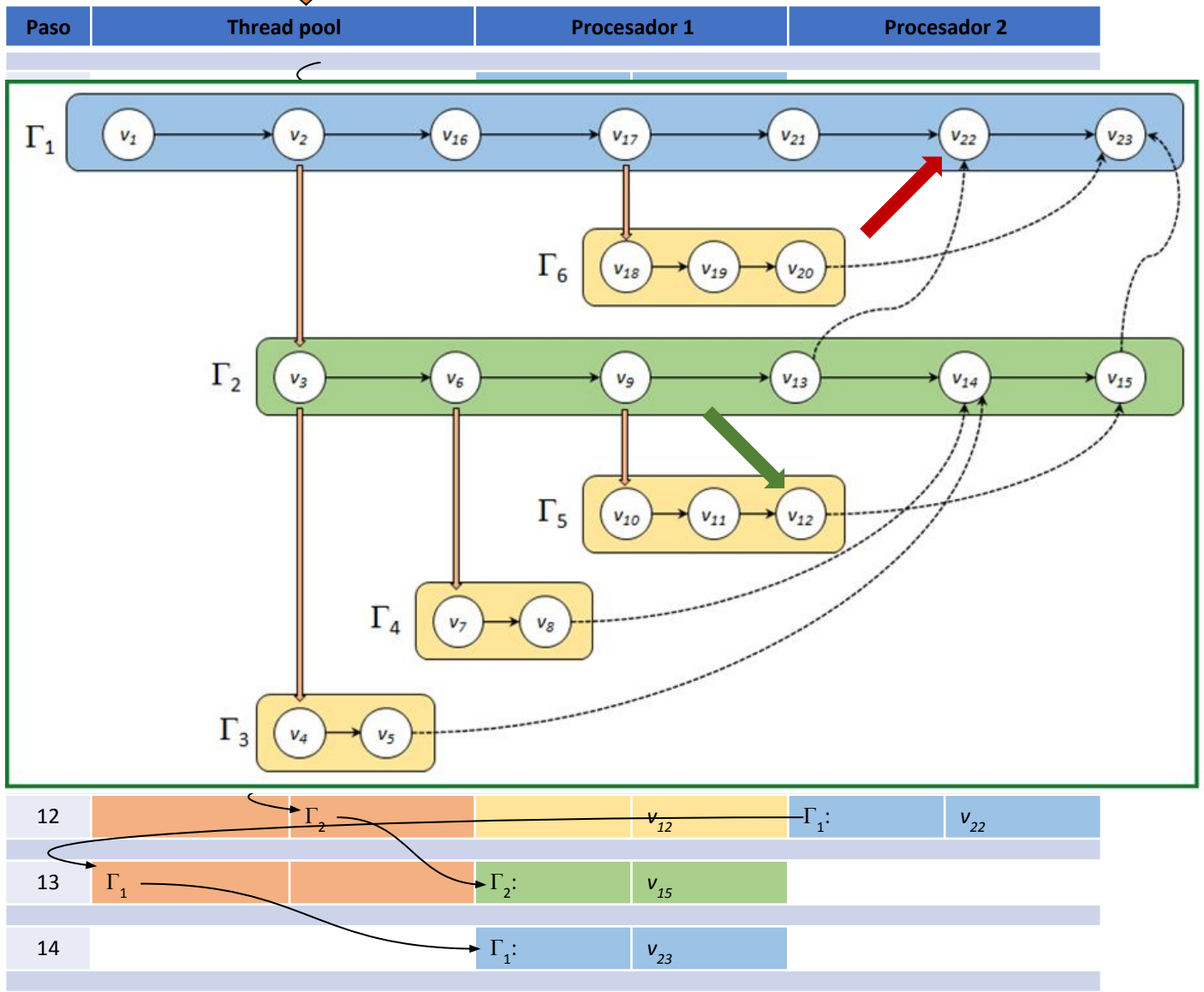
El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.

Paso	Thread pool	Procesador 1	Procesador 2
1		Γ_1: v_1	
2		v_2	
3		Γ_2: v_3	Γ_1: V_{16}
4	Γ_2	Γ_3: v_4	V_{17}
5	Γ_1 Γ_2	v_5	Γ_6: V_{18}
6	Γ_1	Γ_2: v_6	V_{19}
7	Γ_1 Γ_2	Γ_4: v_7	V_{20}
8	Γ_2	v_8	Γ_1: v_{21}
9	Γ_1	Γ_2: v_9	
10	Γ_1	Γ_5: v_{10}	Γ_2: v_{13}
11	Γ_1	v_{11}	v_{14}



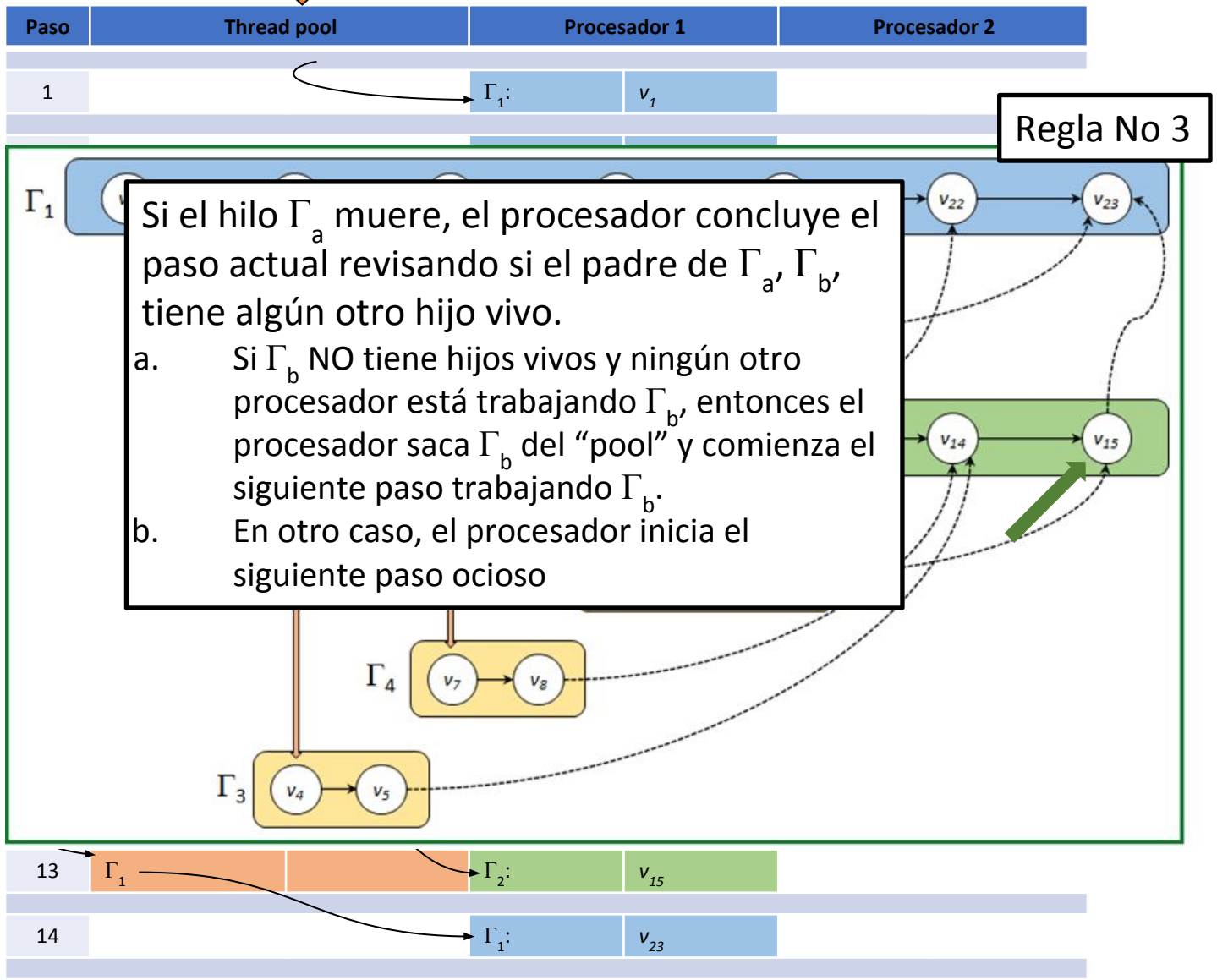
Dos procesadores con el algoritmo “Busy-Leaves”

El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.



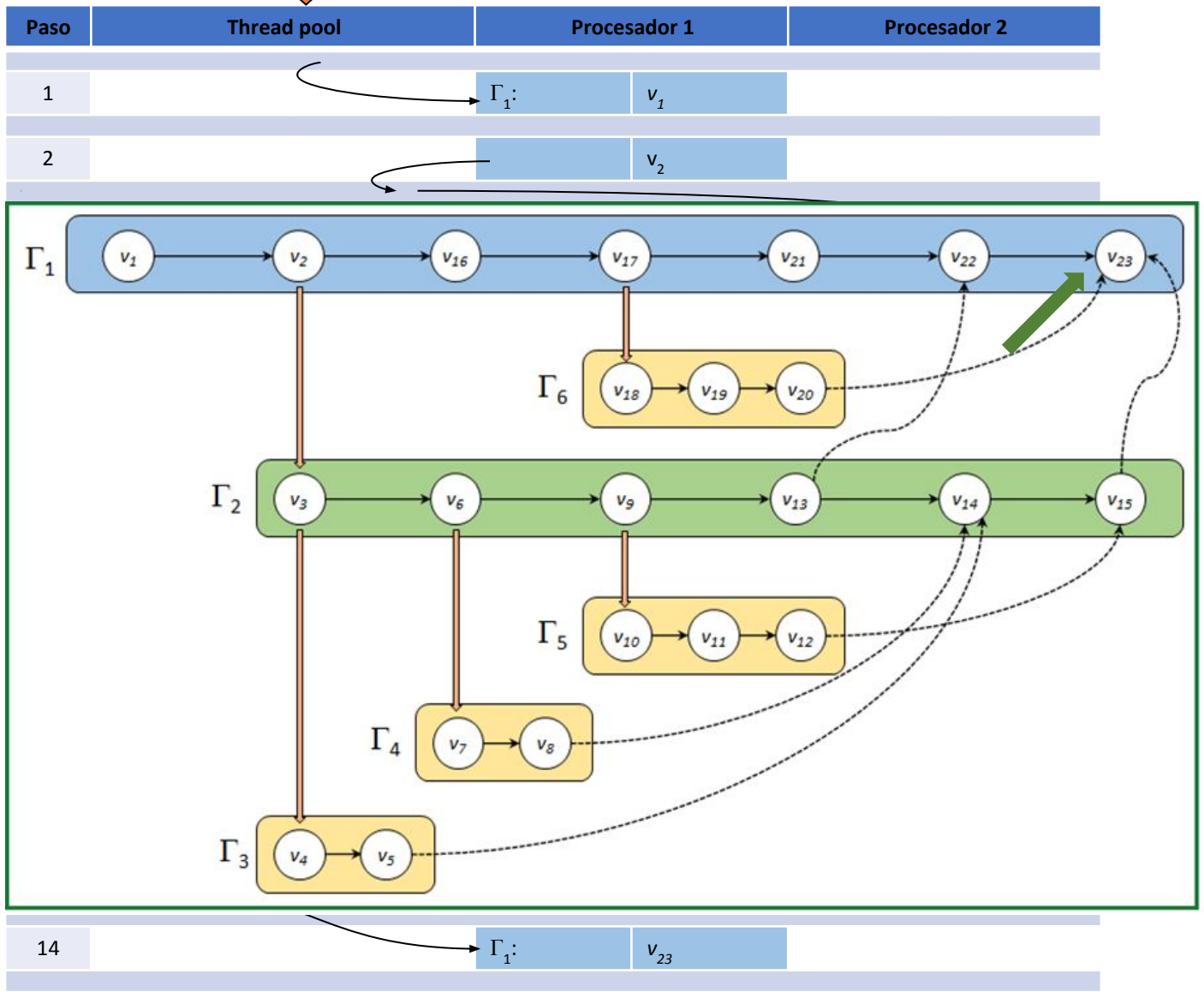
Dos procesadores con el algoritmo “Busy-Leaves”

El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.



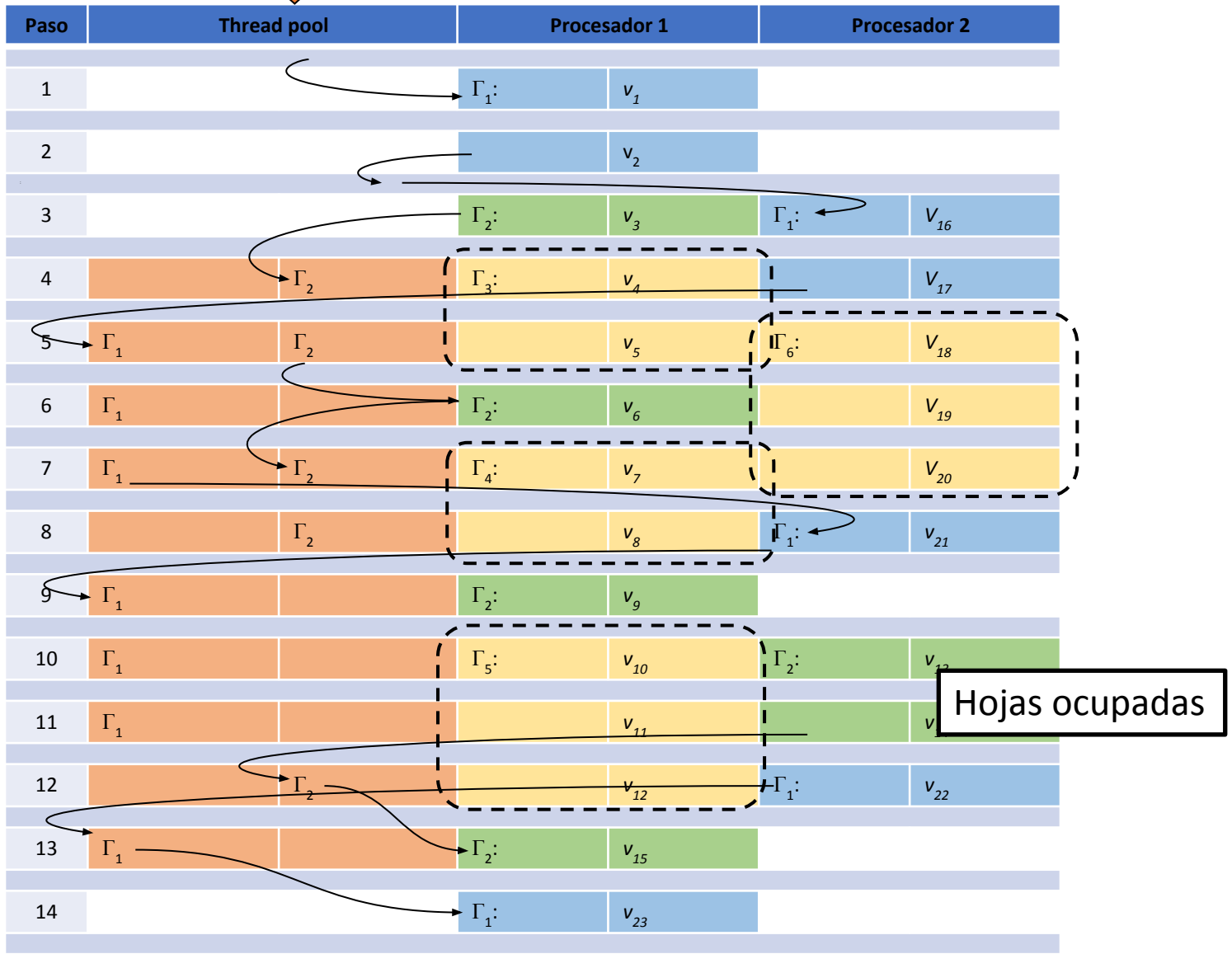
Dos procesadores con el algoritmo “Busy-Leaves”

El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.



Dos procesadores con el algoritmo “Busy-Leaves”

El algoritmo “busy-leaves” mantiene todos los hilos “vivos” en un único ***thread pool*** que está disponible de forma uniforme para todos los procesadores.



Propiedad de hojas ocupadas

- Además de ser una planificación codiciosa, para una computación estricta, la planificación computada por el algoritmo de busy-leaves, mantiene la propiedad de hojas ocupadas:
  - En cada paso durante la ejecución, cada “hilo hoja” en el sub-árbol de generación tiene un procesador trabajando sobre él.
- Se define el sub-árbol de generación en un paso t como la porción del árbol de generación que consta solo de aquellos hilos que están vivos en el paso t.
- Para re-establecer la propiedad de hojas ocupadas, en cada paso, cualquier hilo vivo, que no tenga descendientes vivos tiene un procesador trabajando en él.

Lema 2

- Para cualquier computación multihilo con un tamaño de pila S_1, cualquier planificación de ejecución χ que mantiene la propiedad de hojas ocupadas (busy-leaves) utiliza un espacio acotado por $S(\chi) \leq S_1 P$.

Prueba Lema 2

- La propiedad de hojas ocupadas implica que en todos los pasos t, el ***sub***árbol de generación tiene a lo sumo P hojas.
- Para cada hoja, el espacio utilizado por esta y todos sus ancestros es a lo sumo S_1 y por tanto el espacio utilizado en cualquier paso t es a lo sumo $S_1 P$

Teorema 3

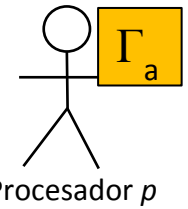
- Para cualquier número P de procesadores y cualquier computación multihilo estricta con work T_1, Span T_∞, y un tamaño de pila S_1, el algoritmo de hojas ocupadas computa un planificador de ejecución χ para P procesadores cuyo tiempo de ejecución satisface $T(\chi) \leq T_1/P + T_\infty$ y cuyo espacio satisface $S(\chi) \leq S_1 P$

Prueba Teorema 3

- El límite en el tiempo sigue directamente del teorema de planificación codiciosa (Teorema 1) debido a que el algoritmo de hojas ocupadas computa con un planificador codicioso
- El límite en el espacio se sigue del lema 2 si se puede mostrar que el algoritmo de hojas ocupadas mantiene la propiedad de hojas ocupadas.
- Se prueba haciendo una inducción sobre el número de pasos.

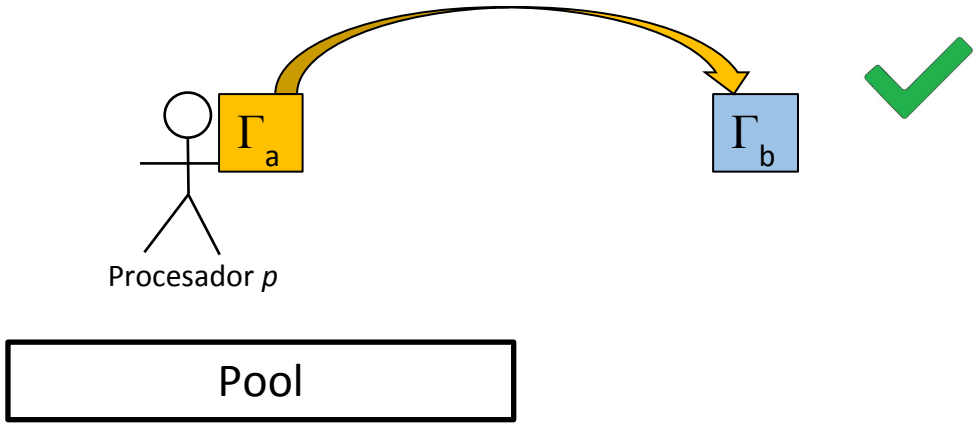
Prueba Teorema 3

- En el primer paso del algoritmo, el sub-árbol de generación contiene solo al hilo raíz, que en ese momento es una hoja, y algún procesador está trabajando sobre este hilo.
- Ahora se debe probar que las tres reglas del algoritmo preservan la propiedad de hojas ocupadas.
- Recordemos que: cuando un procesador tiene un hilo Γ_a, y lo está trabajando, este procesador ejecuta instrucciones de ese hilo hasta que este
  1. Genere otro hilo (spawn)
  2. Se postergue (stall)
  3. Muera (die)



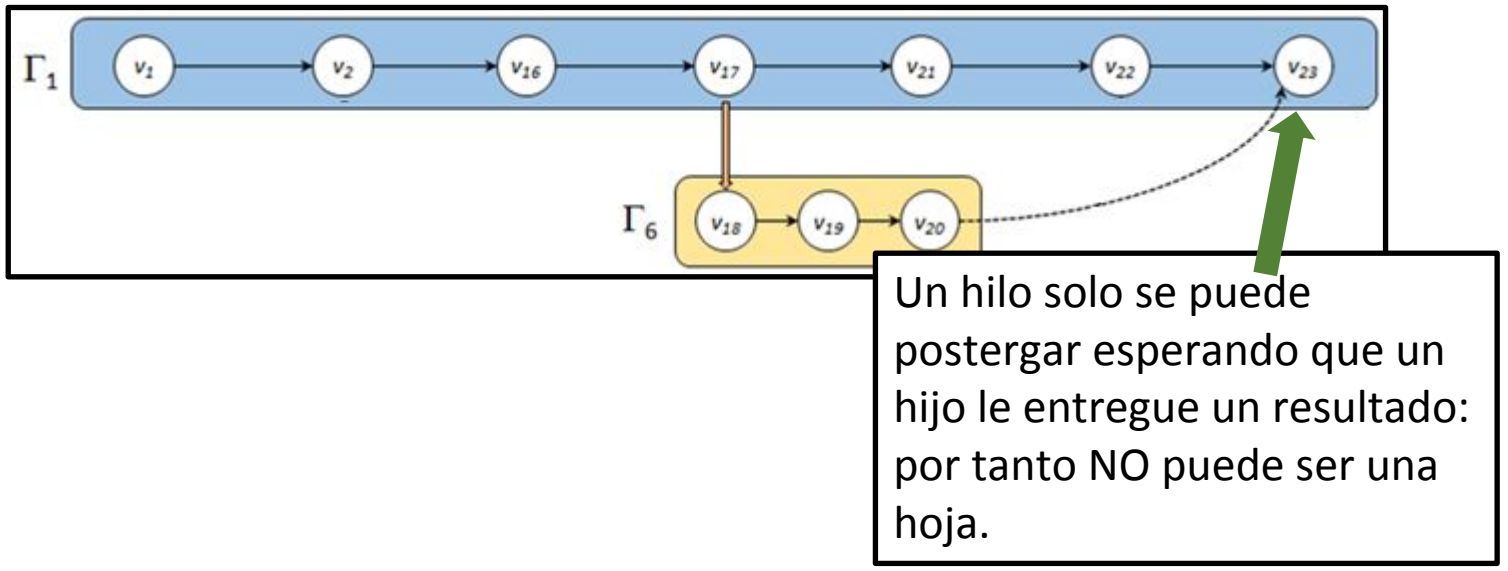
Prueba Teorema 3

- **Regla 1:** Si Γ_a crea un hijo Γ_b, entonces Γ_a NO es una hoja [aunque antes lo hubiera sido, ahora es padre] pero Γ_b SÍ es una hoja. En este caso el procesador trabaja sobre Γ_b, de tal forma que la nueva hoja esta ocupada.



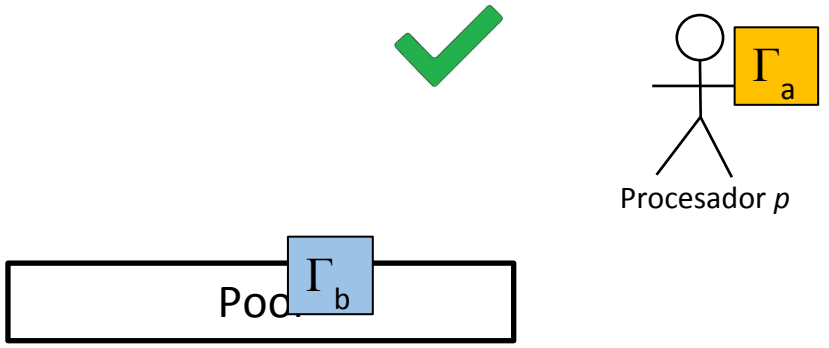
Prueba Teorema 3

- **Regla 2:** Si Γ_a se posterga, entonces Γ_a NO puede ser una hoja ya que esta es una computación estricta y la dependencia no resuelta debe ser causada por algún descendiente.



Prueba Teorema 3

- **Regla 3:** Si Γ_a muere, entonces su padre Γ_b vuelve a ser una hoja. En este caso el procesador regresará a trabajar con Γ_b a menos que otro procesador ya lo esté haciendo, entonces se garantiza que la “nueva” hoja Γ_b esté ocupada.



Escalabilidad del algoritmo de hojas ocupadas

- Hasta aquí ya sabemos que una computación multihilo estricta tiene una planificación de ejecución eficiente [y sabemos como encontrarla]
- Pero se puede ir más lejos
- Las planificaciones de ejecución deben ser computadas eficientemente “online” y aunque el algoritmo de hojas ocupadas computa de manera eficiente planificaciones de ejecución y puede operar “online”, no lo hace de manera eficiente [excepto en casos de pequeña escala con multiprocesadores simétricos].
- Esta falta de escalabilidad es una consecuencia de utilizar un “pool” de hilos centralizados en el cual todos los procesadores deben “competir” por acceder.
- La siguiente sección del artículo se presenta un algoritmo de planificación “online” distribuido y en las secciones que siguen se prueba que es eficiente y escalable.

Referencia principal

- Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, 1999.

Gracias